

融合多模态信息的简易自行瞄准装置

摘要:

本系统以 TI MSPM0G3507 单片机 STM32F103C8T6 构成的最小系统为控制核心，由 405nm 蓝紫光激光模组、二维舵机云台、K230 开发板等模块构成，达到了高速高精度且模式可调的视觉打靶与灰度循迹的要求。

系统功能由按键控制，还可对测量结果实时显示。K230 视觉模块通过引入矩形透射变换方法追踪矩形目标靶，mAP50 在 0.95 以上，显著优于基于 LAB 阈值的色块追踪方案，帧率可维持在 20 以上；舵机由 STM32F103C8T6 单片机控制，根据 K230 传回的靶心坐标进行精准旋转，将打靶误差缩小到 2cm 以内；小车以 MSPM0G3507 单片机为主控，通过灰度传感器与 PID 控制算法对行驶过程中产生的偏差进行补偿，将循迹误差控制在 1cm 以内，实现了高精度的灰度循迹。

系统通过结合多种传感器，融合多模态信息，最终达到了低成本高效率的任务完成效果。

关键词：MSPM0G3507；矩形透射变换；PID 算法；灰度循迹；多模态信息

一、系统方案设计与论证

1. 方案分析与选择

1.1 控制芯片方案：TI MSPM0G3507 与 STM32F103C8T6 分工协作。TI MSPM0 系列是题目指定用于小车巡迹和电机控制的芯片，其在低功耗、外设集成度上适配小车控制需求，虽电路设计有一定复杂度，但符合竞赛要求；STM32F103C8T6 处理能力较强，适合用于舵机控制等瞄准模块的复杂运算，相较于仅用 TI 芯片，可减轻其负载，提升系统稳定性，最终选定此分工方案。

1.2 循迹方案：采用七路灰度传感器。相较于其他传感器，七路灰度传感器能更精准地识别 $1.8\text{cm} \pm 0.2\text{cm}$ 宽的黑色边线，通过多组数据对比，可快速判断小车偏离轨迹情况，及时调整方向，满足小车沿正方形轨迹逆时针寻迹行驶的需求。

1.3 靶心坐标定位方案：通过 K230 视觉模块获取目标靶的图像，利用矩形透射变换的方法寻找靶子的四个角点，进而得到靶心。这一方案在定位精度上比基于色块追踪的方案更好，最终帧率能达到 20 以上，能够满足视觉测量的要求。

1.4 脱机阈值调整方案：在视觉识别方面，通过 GPIO 口向 K230 视觉模块发送按键信号，通过三个按键进行阈值调整。当按键按下，可以选择调整的参数、参数需要上升还是下降，并通过按键触发参数调整。在具体任务的选择上，可以通过按键向 MSPM0G3507 发送信息，从而实现小车的任务选择以及相关参数的调整，原理与视觉识别的参数调整类似。这种方案通过人机交互的方式方便了用户的使用。

1.5 瞄准与定位方案：结合 K230 视觉模块和二维云台。由 K230 采集目标靶图像，进行图像处理，通过矩形透射变换的方法找到目标靶轮廓，并据此确定靶心，将数据传输给 STM32F103C8T6，控制二维云台调整 405nm 蓝紫光激光模组方

向，实现瞄准和画圆功能，该方案能精准响应目标位置变化。

1.6 总体方案描述

系统由自动寻迹小车和瞄准模块两部分组成。自动寻迹小车以 TI MSPM0G3507 为控制核心，通过七路灰度传感器识别黑色轨迹，控制电机驱动小车沿 $100\text{cm} \times 100\text{cm}$ 的正方形轨迹逆时针行驶并直线转弯，行驶圈数可在 1-5 之间设定。瞄准模块以 STM32F103C8T6 为控制核心，搭载 K230 视觉模块和二维云台，摄像头获取目标靶图像后，经 K230 处理得到靶心位置信息，由 STM32F103C8T6 控制二维云台调整 405nm 蓝紫光激光模组，使激光光斑落在目标靶指定位置。两个模块分别由独立开关控制电源，并用发光管显示供电状态。

二、理论分析与计算

1. 测量原理与计算

1.1 循迹测量：七路灰度传感器输出不同电压值反映轨迹线与背景的灰度差异，TI MSPM0G3507 采集这些电压值，通过预设阈值判断小车位置，计算偏离量，进而控制电机转速差实现转向，使小车沿轨迹行驶。

1.2 瞄准测量：摄像头采集靶面图像，K230 对图像进行灰度化、边缘检测等处理，识别出靶心及红色圆弧，通过图像坐标与实际坐标的换算关系，计算激光模组需要调整的角度，STM32F103C8T6 控制舵机转动相应角度，实现瞄准。

2. 误差分析

2.1 靶心位置测量误差：由于 K230 视觉模块的精度不足以及场地光照干扰等原因，通过矩形透射变换的方法计算得到的靶心坐标存在误差。

2.2 灰度循迹误差：通过七路灰度传感器获取场地上黑色轨迹的位置后，由 MSPM0G3507 进行处理，通过 PID 算法调整运动策略，使小车中心线与轨迹中心线相重合。这一过程中，受到灰度传感器排列密度以及 PID 参数等影响，小车中心线往往难以与轨迹中心线完全重合。

2.3 激光打靶误差: 由于各种原因,K230 视觉模块测得的靶心坐标存在一定误差,再加上 STM32F103C8T6 的 PID 参数不够精确等,激光往往难以做到与靶心重合。

三、参数计算，系统设计与实现

1. 电路设计

系统的灰度传感器电路中,LED 发光二极管发射灯光,投射到路面后反射。以接收管接收反射光,浅色路面反射强,接收管电阻降低、输出电压升高;深色路面则相反。接收管输出的微弱电信号,经过由电阻构成的分压电路等处理,转化为适合后续电路处理的电压信号。电压信号输入到 LM393 电压比较器,将输入电压与参考电压比较。当输入电压高于参考电压,比较器输出高电平,否则输出低电平。比较器输出的高低电平信号传输给 STM32 微控制器。微控制器预先设定了黑白线判断阈值,根据接收到的电平信号判断传感器是否在黑线上。

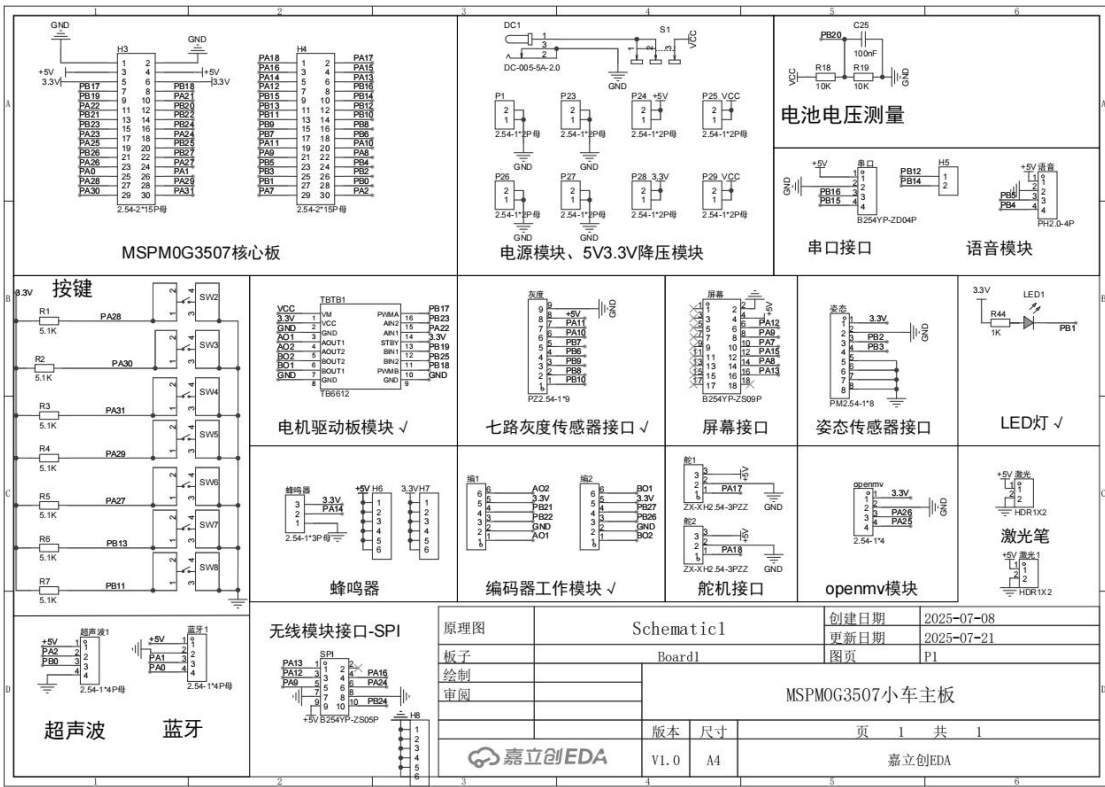


图 3-1：MSPM0G3507 小车主板

系统由 MSPM0G3507 作主控芯片,驱动模块为 TB6612FNG,另外还具有电源模块、OLED 模块、按键模块等。

幅输出，防止失控。

3) 电机驱动

通过给左右两个电机输入不同的 PWM 信号，可以控制两个电机的转速。通过电机之间的差速，可以实现小车方向的改变。

4) PID 算法

因为此次比赛赛道不大并且没有很长的直线，在此前本组曾尝试过使用速度环与转向环并联的方式，但是调试效果并不理想，大多数时候速度环还未完成完成加速时就已经来到了下一个弯道，小车跑起来的效果没有开环跑的效果好，所以决定在本次比赛当中采用开环跑。

PID 是最经典的控制算法之一，广泛应用于工业控制、机器人、自动化等领域。时至今日，依然有大约 90%的控制回路拥有 PID 结构。其流程图如下所示：

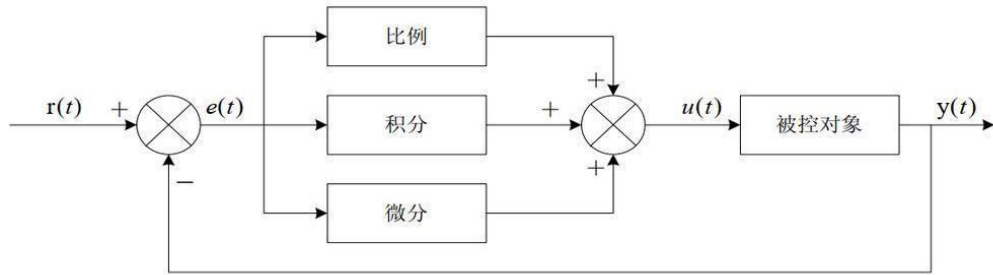


图 3-3: PID 算法流程图

在模拟控制系统中，PID 控制器的输出 $u(t)$ 表示为：

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1)$$

位置式 PID，在前面谈到加权平均计算小车位置，当误差为 0 时证明小车走在路线中央所以将目标误差设置为 0，再将 PID 的输出与电机的基础占空比结合起来。将输出限幅过后再赋值给电机，确保电机不会失控。

在 PID 算法中，P、I、D 分别表示比例、积分和控制，公式如下：

$$u_p = K_p e(k), u_i = K_i T_s \sum e(k), u_d = K_d \frac{e(k) - e(k-1)}{T_s} \quad (2)$$

5) 靶心定位

系统的靶心定位分为阈值调整阶段和目标追踪阶段，由按键控制。

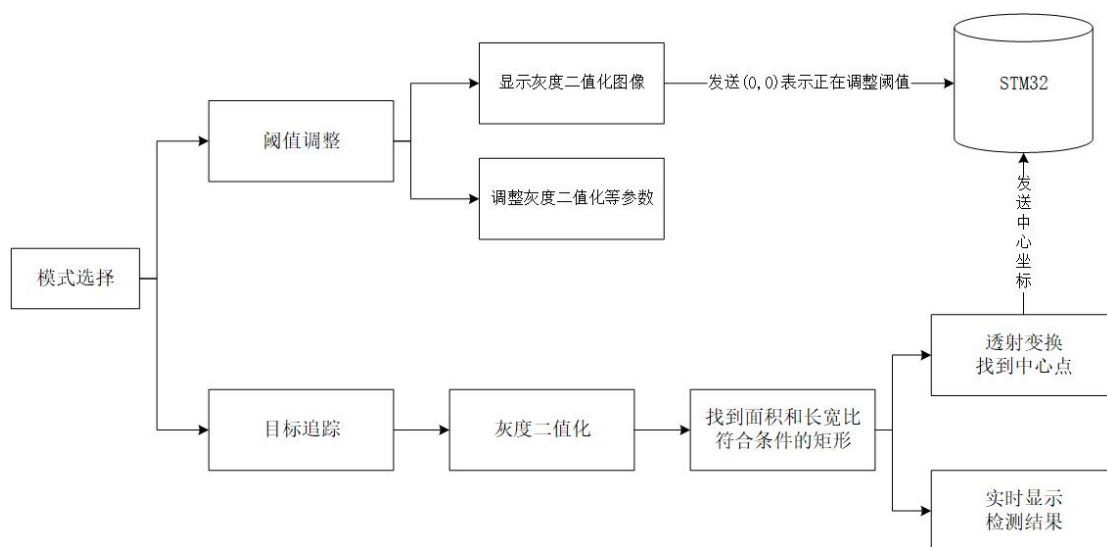


图 3-4：靶心定位流程图

在阈值调整阶段，K230 屏幕上会显示摄像头的灰度二值化图像，用户可以通过连接到 K230 GPIO 的按键来实时调整灰度二值化的上下阈值等参数，屏幕上会叠加显示当前调整的参数等信息，方便用户直观地观察二值化效果，并根据实际环境光照和目标颜色进行优化。

在目标追踪阶段，程序会利用调整后的灰度二值化阈值对图像进行处理，识别出图像中的矩形目标，进一步筛选出符合特定面积和宽高比的最小矩形。对于识别到的矩形，代码会进行透视变换，将矩形映射到一个虚拟的平面，并在虚拟平面上绘制一个校正后的圆形，圆心即为这个矩形的中心点。最终，矩形中心点的坐标会通过串口通信的方式发送给 STM32F103C8T6，实现目标的实时追踪和定位，并让云台旋转实现激光打靶。

四、测试方案与测试结果

1. 测试方案

1) 测试仪器

除作品上的 MSPM0G3507 等元器件，还包括数字万用表、示波器、秒表、卷尺。

2) 测试环境

测试场地为传统实验室环境，背景为白色墙面以及白色桌子等实验室常见摆放物，周围无强光干扰，无可能对作品产生特殊影响的物品。靶面靠墙竖立，行驶场地采用白色哑光喷绘布制作，水平铺设于整洁地面。

3) 测试方法

针对第一个基本要求，将瞄准模块电源开关断开后，将小车放在场地上，通过按键设置小车需要完成的任务以及相关参数。按下开关按键，小车可以沿行驶轨迹自动寻迹行驶，行驶圈数 N 可在 1 到 5 之间设定。通过秒表测量行驶时间，得到 $t \leq 20s$ 。

针对第二个基本要求，将小车放置在场地上，位置和姿态由自己决定。按下按键启动瞄准模块，由秒表测量激光模块发射激光击中靶心所需时间，发现小于 2s。接着，利用卷尺测量光斑痕迹距靶心最大距离，得到 $D1 \leq 2cm$ 。

针对第三个基本要求，将小车放置在行驶轨迹上的指定位置，瞄准方向任意。启动瞄准模块后，利用秒表测量作品自动瞄准发射激光击中靶心所需时间，得到 $t \leq 4s$ ，同时用卷尺测量光斑痕迹距靶心最大距离，得到 $D1 \leq 2cm$ 。

针对发挥部分，将小车放置在 AB 段轨迹上，前沿投影与 AC 线对齐，按下按键，启动小车和瞄准模块，让小车在指定时间内沿轨迹自动寻迹行驶指定圈数，同时观察标靶，利用卷尺测量运动期间激光笔是否连续发光射向靶心且误差较小，并利用秒表测量行驶所需时间。

2. 系统性能指标测试与分析

针对视觉打靶，在不同位置、不同角度、不同光照条件下测量其对于目标靶的识别效果。在识别效果上，K230 采用的矩形透射变换方法在矩形识别任务的准确率、精确率、召回率和 F1 的得分都接近 1.0，优于色块追踪的 0.90；在靶心定位任务中，矩形透射变换测得的靶心与实际靶心的误差在 5 个像素以内，且对

不同环境表现出较强的鲁棒性，能够满足视觉任务要求。

针对灰度循迹，将小车放置在白色哑光喷绘布制作的场地上，在场地的不同位置进行测试，同时改变场地的光照条件以及小车的放置角度，发现灰度传感器在寻找黑色轨迹任务中精度高、响应快，通过与 PID 控制算法的配合，可以实现小车行驶误差的快速调整。

针对小车行驶速度，通过调整 MSPM0G3507 的 PWM 输出等参数，我们的小车能够以较快的速度完成任务，行驶一圈所消耗的时间可以轻松控制在 20s 以内。同时，小车前方安装的万向轮方便了小车的快速转向，节约了任务完成所需时间，能够高质量完成任务要求。

3. 测量结果分析

本系统采用了一种融合多模态传感器的小车与二位云台协同作业方法，以 MSPM0G3507 为主控芯片，以 STM32F103C8T6 单片机与 K230 视觉模块为辅助，能够实现高速高精度的灰度循迹与视觉打靶。在目标靶的检测过程中，模型的 mAP50 达到 0.95 以上，测得目标靶靶心坐标与实际坐标的误差不超过 5 个像素。在灰度循迹过程中，小车中心线与轨迹中心线几乎完全重合，同时保证以较快速度运行。

五、总结

本系统通过理论分析得到合理的方案，以 MSPM0G3507 为控制核心，结合 STM32F103C8T6 与 K230 视觉模块，设计并实现了小车循迹和激光自行瞄准装置，完成了小车逆时针沿正方形直角转弯和激光自行瞄准，并达到较高的精度与较快的速度，各项指标符合设计任务要求。

六、附录

```
//移动平均滤波
error_history[history_index] = position_error; // 存入当前误差
history_index = (history_index + 1) % FILTER_WINDOW_SIZE; // 循环队列

int32_t filtered_error_sum = 0;
for(int i=0; i<FILTER_WINDOW_SIZE; i++) {
    filtered_error_sum += error_history[i];
}
position_error = filtered_error_sum / FILTER_WINDOW_SIZE; // 滤波后误差

count = 0; // 计次清零
```

图 7-1: 移动滤波的代码实现

```
33  /*
34  //左边电机设置速度
35  void Motor_SetSpeed(float left,float right)
36  {
37      if (left >= 0) //如果设置正转的速度值
38      {
39          GPIO_SetBits(GPIOA, GPIO_Pin_9); //PA4置高电平
40          GPIO_ResetBits(GPIOA, GPIO_Pin_10); //PA5置低电平, 设置方向为正转
41          PWM_SetCompare3(left); //PWM设置为速度值
42      }
43      else //否则, 即设置反转的速度值
44      {
45          GPIO_ResetBits(GPIOA, GPIO_Pin_9); //PA4置低电平
46          GPIO_SetBits(GPIOA, GPIO_Pin_10); //PA5置高电平, 设置方向为反转
47          PWM_SetCompare3(-left); //PWM设置为负的速度值, 因为此时速度值为负数, 而PWM只能给正数
48      }
49
50      if (right >= 0) //如果设置正转的速度值
51      {
52          GPIO_SetBits(GPIOA, GPIO_Pin_15); //PA4置高电平
53          GPIO_ResetBits(GPIOA, GPIO_Pin_12); //PA5置低电平, 设置方向为正转
54          PWM_SetCompare4(right); //PWM设置为速度值
55      }
56      else //否则, 即设置反转的速度值
57      {
58          GPIO_ResetBits(GPIOA, GPIO_Pin_15); //PA4置低电平
59          GPIO_SetBits(GPIOA, GPIO_Pin_12); //PA5置高电平, 设置方向为反转
60          PWM_SetCompare4(-right); //PWM设置为负的速度值, 因为此时速度值为负数, 而PWM只能给正数
61      }
62  }
```

图 7-2: 电机驱动的代码实现

```
5  float Angle_PID(float Aim_Angle,float Now_Angle)
6  {
7      static float err,last_err;
8      static float p_out,d_out,out;
9      last_err = err;
10     err = Aim_Angle - Now_Angle;
11     p_out = B_KP * err;
12     d_out = B_KD * (err-last_err);
13     out = p_out + d_out;
14     if(out > outmax) out = outmax;
15     if(out < -outmax) out = -outmax;
16     return out;
17 }
```

图 7-3: PID 算法调整小车行驶角度

```

55 |
56 | void Motor_control(float outmax, float left, float right)
57 | {
58 |     left = left > outmax ? outmax : left;
59 |     left = left < -outmax ? -outmax : left;
60 |     right = right > outmax ? outmax : right;
61 |     right = right < -outmax ? -outmax : right;
62 |     Motor_SetSpeed(left, right);
63 | }
64 |
65 |
66 |

```

图 7-4：通过电机调整小车行驶速度

```

63 |
64 | void TIM2_IRQHandler(void)
65 | {
66 |     if (TIM_GetITStatus(TIM2, TIM_IT_Update) == SET)
67 |     {
68 |         if (Car_Move_Flag) { // 只有在发车标志为1时才执行控制逻辑
69 |             ;
70 |             Infrared BitGet();
71 |             // 计算转向环PID
72 |             steering_output = Angle_PID(0, position_error); // 来自Infrared.c
73 |             left_duty = initialsp + steering_output;
74 |             right_duty = initialsp - steering_output;
75 |             Motor_control(99, left_duty, right_duty);
76 |         }
77 |
78 |         // 清除中断标志
79 |         TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
80 |     }
81 | }
82 |

```

图 7-5：PWM 限幅后再赋值给小车

```

def pack_data(x, y):
    """
    数据打包函数，把坐标转换成串口通信的格式。
    格式为：起始字节 + X高字节 + X低字节 + Y高字节 + Y低字节 + 结束字节
    """
    # 使用固定的传感器分辨率 320x240
    sensor_width = 320
    sensor_height = 240

    x = max(0, min(x, sensor_width - 1))
    y = max(0, min(y, sensor_height - 1))

    data = bytearray([
        0x2C,
        (x >> 8) & 0xFF,
        x & 0xFF,
        (y >> 8) & 0xFF,
        y & 0xFF,
        0x5B
    ])
    return data

```

图 7-6：将视觉检测结果打包方便串口通信传输数据

```

# 构建虚拟矩形（根据方向调整）
if is_horizontal:
    virtual_rect = [(0, 0), (RECT_WIDTH, 0), (RECT_WIDTH, RECT_HEIGHT), (0, RECT_HEIGHT)]
else:
    virtual_rect = [(0, 0), (RECT_HEIGHT, 0), (RECT_HEIGHT, RECT_WIDTH), (0, RECT_WIDTH)]

# 计算校正半径（基于实际宽高比）
# 注意：这里使用调整后的 BASE_RADIUS
if is_horizontal:
    radius_x = BASE_RADIUS
    radius_y = BASE_RADIUS / actual_aspect
else:
    radius_x = BASE_RADIUS * actual_aspect
    radius_y = BASE_RADIUS

# 计算虚拟矩形中心
virtual_center = (RECT_WIDTH/2, RECT_HEIGHT/2) if is_horizontal else (RECT_HEIGHT/2, RECT_WIDTH/2)

# 在虚拟矩形中生成椭圆点集（映射后为正圆）
virtual_circle_points = []
for i in range(POINTS_PER_CIRCLE):
    angle = 2 * math.pi * i / POINTS_PER_CIRCLE
    x = virtual_center[0] + radius_x * math.cos(angle)
    y = virtual_center[1] + radius_y * math.sin(angle)
    virtual_circle_points.append((x, y))

# 计算透视变换矩阵并映射坐标
matrix = get_perspective_matrix(virtual_rect, sorted_corners)
if matrix:
    mapped_points = transform_points(virtual_circle_points, matrix)
    int_points = [(int(round(x)), int(round(y))) for x, y in mapped_points]

    # 绘制圆形
    for (x, y) in int_points:
        display_img.draw_circle(x, y, 2, color=(255, 0, 255), thickness=2)

    # 绘制圆心
    mapped_center = transform_points([virtual_center], matrix)
    if mapped_center:
        cx, cy = map(int, map(round, mapped_center[0]))
        display_img.draw_circle(cx, cy, 3, color=(0, 0, 255), thickness=1)
        data_to_send = pack_data(int(round(cx)), int(round(cy)))
        uart.write(data_to_send) # 串口通信发送中心坐标
        print(f"校正后圆心坐标（矩形中心）：{[(int(round(cx)), int(round(cy)))]}")

```

图 7-7：通过矩形透射变换寻找靶心

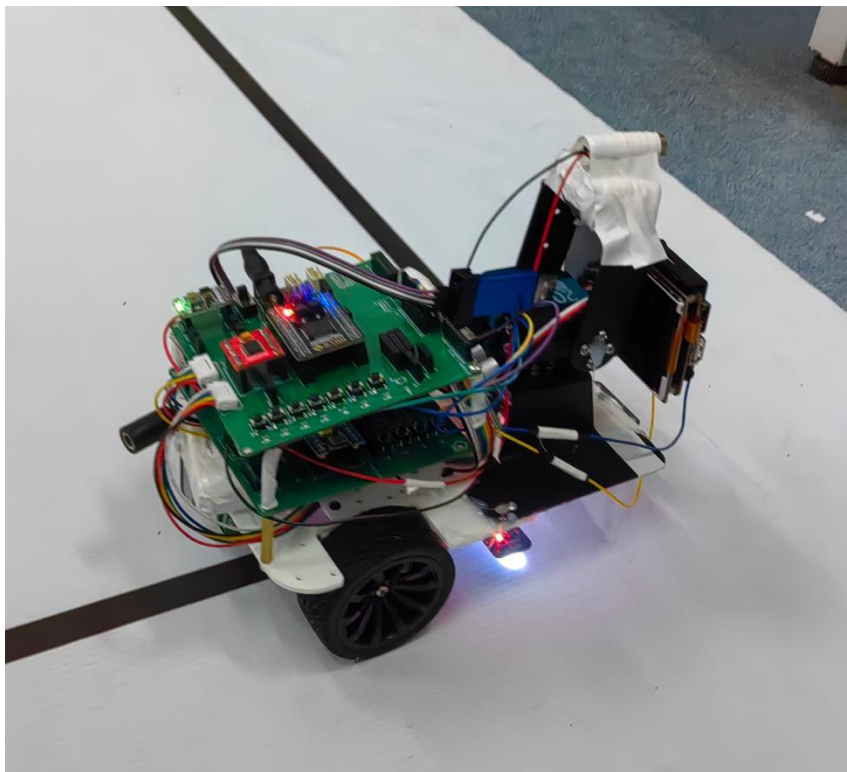


图 7-8：作品实物图